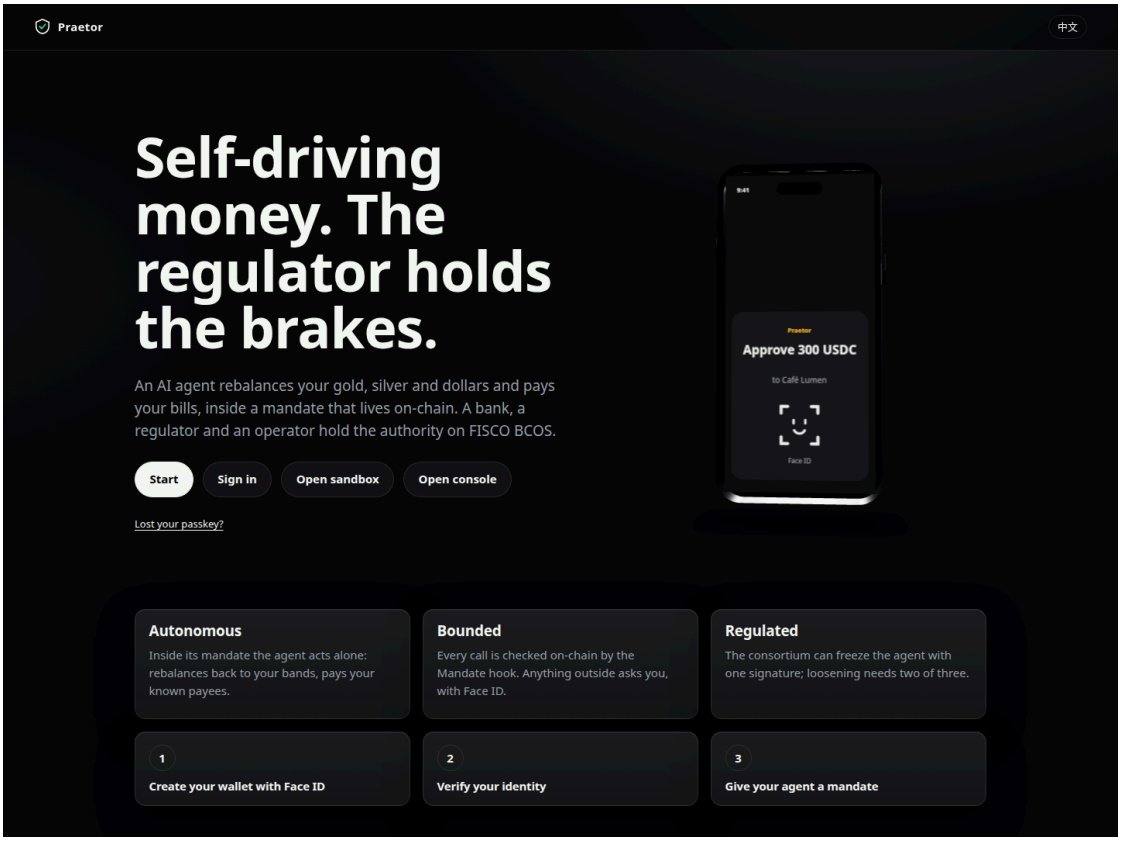
Praetor

> **Self-driving money; the regulator holds the brakes.**

FinTechathon 2026 (Shenzhen, WeBank x Tsinghua PBCSF), International Track, Topic F "Agent Wallet". 中文: README.zh-CN.md.

Try it: https://app.praetorwallet.com (Base Sepolia, testnet only; onboard at `/start` with a passkey; 1 USDC on chain stands for 100 in the story, and demo funds are small, on request from the team). Not open to visitors there, by design: the consortium console (staff devices approved by an enrolled staff member, D99) and WeChat (testers of the mini program's test account only), so the freeze and the WeChat approval are shown in the video and in docs/EVIDENCE-SERVER.md; the one-click sandbox runs only on a local fork (`make up`).

An AI agent manages your portfolio (USDC, tokenized gold and silver) and pays your bills on its own, from an ERC-4337 smart account on **Base Sepolia**. Every call the account makes passes an on-chain **Mandate hook** that applies your rules and the bank's caps, default deny; anything unusual asks you for Face ID (a passkey, or SOTER inside WeChat). A **FISCO BCOS** consortium of a bank, a regulator and an operator holds the authority: identity (WeIdentity), approved assets, sanctions, the agent's status and an append-only audit log. Its decisions reach Base as member signatures the hook verifies itself, carried by **WeCross** with a relayer as watchdog. The agent's key is held by a separate signing service in a **TEE**: on the public server, a Phala Cloud CVM in Intel TDX (dstack) whose quote the bank verifies; local and fork runs use the dstack simulator. One regulator signature freezes the agent at contract level; loosening needs two of three.



The demo in five lines

1. **Onboard with Face ID**: a passkey creates the smart account, demo KYC at the bank, the agent's key is derived by the TEE signing service (the dstack simulator outside Phala Cloud) and checked by the bank, the mandate is signed by you and co-signed by the bank.
2. **Gold jumps 20 %** (demo price, on the local fork): the agent wakes up and sells about 650 USD of tXAU to get back into its band, alone (T1).
3. **"Pay 300 to a new payee"**: refused on-chain (unknown payee), you approve with your passkey (or Face ID in WeChat, M14), then it pays (T2).
4. **A prompt injection** asks the agent to pay a sanctioned address: the model declines; the same payment signed with the agent's own key is refused on-chain, and `RefusalLog` records the reason the hook computed (SANCTIONED).
5. **The regulator freezes the agent** from the console with one signature: FISCO, WeCross, Base; the next action is refused FROZEN; unfreezing needs two members. Then the audit chain verifies against FISCO and its Base anchor, and exports as an ISO 20022 camt.053 statement.

How it is bounded

Tier	Who	Example
T0 Never	nobody: the hook refuses	over the bank's caps, sanctioned payee, hidden call, while frozen
T1 Autonomous	the agent alone	pay a known payee under the auto cap, rebalance inside the bands, a due standing order
T2 Human	the owner: passkey or SOTER	unknown payee, over the auto cap or a budget
T3 Owner + bank	the owner and the bank (the owner alone to make things stricter)	change the mandate, standing orders, the agent key
T4 Consortium	1 of 3 to tighten, 2 of 3 to loosen, the bank for caps and KYA	freeze, sanctions, assets, heartbeat

Read the architecture for the picture and the algorithms, and the security self-assessment for the invariants and the tests that prove them.

Quick start (local stack)

Prerequisites: Linux x86_64 (tested on Ubuntu 24.04) with Docker (Compose v2), Node 22 with pnpm 10.29.1, Foundry 1.7.1, 16 GB of RAM recommended (the production server's size). Java runs only inside pinned images.

```
git clone --recurse-submodules <repository> praetor && cd praetor   # or: git submodule update --init --recursive
make setup        # commit-msg hook + pnpm install --frozen-lockfile
pnpm keys         # fills every missing key and secret into .env (mode 600); prints public addresses only
make up           # the whole stack: anvil fork of Base Sepolia, 4 FISCO nodes, WeIdentity, WeCross, TEE simulator,
                  # member services, agent, mocks; deploys the contracts on a fresh fork; builds the web app
make test         # guards (em dash, links, attribution), gitleaks, type-check, every unit test, Foundry (needs Docker)
make test-int     # integration tests, one per link of the architecture (needs make up)
make e2e          # flows F1 to F10 through the api and the web app (Playwright), then the 48 h recovery (needs make up)
pnpm seed         # optional: the demo persona Lin, onboarded and funded
```

Then open **https://praetor.localhost**. The certificate comes from Caddy's internal CA: accept the browser warning, or trust the root certificate at `/data/caddy/pki/authorities/local/root.crt` in the `caddy` container. More

targets: `make help` (`make test-contracts-full` for 10,000 fuzz runs, `make test-fork` for the Safe7579 fork tests, `make slither`, `make deploy-contracts`, `make deploy-fisco`).

Repository layout

contracts/base	Base Sepolia contracts (Foundry): MandateHook and its libraries, validators, NoOwner, StandingOrders, PayoutRail, PriceOracle, TreasuryDesk, RefusalLog, Anchor, paymaster, tokens
contracts/fisco	FISCO BCOS contracts: member registry, consortium boards, audit log, Base outbox
contracts/lib	pinned Solidity libraries (git submodules)
packages/	protocol (EIP-712 types, reason codes, golden vectors in TS and Solidity), chain, userop, config, db, fisco-client, member-kit, weid-client
services/	api, mcp, agent-runner, scheduler, bundler, oracle-feeder, notifier, bridge-relayer, bank, regulator, operator, agent-signer and soter-attester (TEEs), identity (WeIdentity), payout-partner and the labelled mocks (Alipay+, WeChat Pay, KYC vendor), fake-llm (tests)
wecross/evm-stub	Praetor's WeCross stub for Base (Java 8)
apps/web	the web app: landing with the 3D phone, onboarding, dashboard, approvals, console, sandbox (EN, 中文)
apps/miniprogram	the WeChat mini program: home, chat, SOTER approvals, WeChat link (EN, 中文)
infra/	compose.yml, compose.prod.yml, live.env, Caddyfile, FISCO, WeIdentity, WeCross and TEE simulator images, deployments
test/	integration (one file per link), e2e (flows F1 to F10), warp (the 48 h recovery), infra checks
scripts/	keys, demo (seed, price shock), deploy (contracts, live funding, prod refusals), evidence (live run), checks (em dash, links, gitleaks, slither)
spec/	the build specification (start with spec/README.md)
docs/	architecture, security pack, deployment, progress, decisions, manual, evidence
spikes/	Phase 0 verification spikes

Documentation

Document	What it covers
docs/ARCHITECTURE.md	The system in one picture, core algorithms, security design, deployment
docs/security/SELF_ASSESSMENT.md	Tiers, invariants I1 to I19 with their tests, threat model, known risks, tool results
docs/security/README.md	The security pack: Slither, dependency audit, TEE checklist, WeCross notes, gas
docs/DEPLOY.md	Deploying to a server and Phala Cloud (Phase C)
spec/README.md	The specification the build follows
docs/PROGRESS.md, docs/DECISIONS.md	Where the build stands; decisions taken since the spec
docs/MANUAL.md	The guided acceptance steps that need a human (phones, WeChat, cloud)
docs/TESTER.md	For a teammate testing on real phones: own stack, WeChat test account, tunnel, what to report
docs/EVIDENCE.md	Transaction hashes of the live run on Base Sepolia (2026-10-18, D96; the M16 deployment, `infra/deployments/base-sepolia-m16.json`), executed and refused, with the FISCO calls of each flow
docs/EVIDENCE-SERVER.md	The public server (2026-10-20, D106): its deployment, the two Phala Cloud CVMs with their app ids and compose hashes, the attester's activation, and the BaseScan links of the WeChat run (refused UNKNOWN_PAYEE, approved with SOTER, executed once)
docs/PREEXISTING.md	Code reused from the team's earlier projects
中文	README.zh-CN.md, docs/zh-CN/

Status

Built and tested on the local fork: the Base and FISCO contracts, the protocol, the bundler and paymaster, the consortium services, WeIdentity identity and KYC, both TEEs on the dstack simulator, the agent with its MCP server, the api, standing orders and payouts, the audit chain with anchors and camt.053, the WeCross transport, the web app with the consortium console and an interim judges' sandbox (D48), the WeChat mini program (logic and api; on phones, see below), agent key rotation and owner recovery (F10), the Guomi profile smoke, the production compose override and its refusal checks. Flows F1 to F10 pass through the api and through the web app (Playwright, virtual authenticator) on fresh forks: the gates of M12 (WeCross as the transport, 2026-10-17), M13 (the web journey, the sandbox and 34 screenshots with axe) and M15 (F10 and the 48 h recovery) passed, the last two on 2026-10-18. The contracts are deployed on Base Sepolia twice: the M16 deployment of 2026-10-18 (MandateHook `0x839E104c8C2df983213a89CAcB58440Da4AdC2A9`, `infra/deployments/base-sepolia-m16.json`), which the evidence run used, and the server's deployment of 2026-10-19 (MandateHook `0x6411cc907Ace53F1Bdf8607a85991d3b38E79a5f`, `infra/deployments/base-sepolia.json`, D101). On Base Sepolia the oracle takes real XAU and XAG prices from Pyth Hermes once every 5 minutes (the fork keeps its seeded demo prices, D89). The red team (spec 08) ran against the local stack on 2026-10-18, and again on 2026-10-19 as the final run before the public server: 85 scripted attacks, 3 on shared state and 12 with the real model, 0 executed (docs/security/redteam.md); the findings of the earlier runs, none of which moved value, are fixed and re-tested (D86 to D88). The deployment rehearsal passed (M18, D90): `make rehearsal` on the fork with `compose.prod.yml`, smoke F1, F5, F8 16/16 and the five prod refusals. On 2026-10-18 the team passed the WeChat steps on an iPhone (link, SOTER Face ID approval, replay refused, regulator freeze, unlink, Chinese UI; docs/MANUAL.md section 5). M16 closed on 2026-10-18 with the live evidence run (D96): on Base Sepolia, with real Pyth prices (tXAU 4,165.33 USD, tXAG 56.99 USD at Friday's close), the real model (DeepSeek through OpenCode) and WeCross as the transport, at `DEMO_SCALE` 100 and within the human's budget (D74). docs/EVIDENCE.md, generated from the chains, lists 10 flows (C0 to F8, F6 twice) with 21 BaseScan links to Base transactions and the FISCO calls: an autonomous T1 payment, a prompt injection to a sanctioned address that the real model declined, with the same payment signed with the agent key refused on-chain (SANCTIONED), a new payee refused (UNKNOWN_PAYEE) then approved with the owner's passkey and executed once, the rent standing order paid through the payout rail (Alipay+ mock), the regulator's freeze delivered through WeCross with an agent payment refused (FROZEN) and the freeze loosened by two members, and audit anchors on Base. F6 failed on its first pass (old agent instructions, issue #4) and was re-run on the same account after D93; both attempts are in the evidence. The teammates' issues #1 to #11 from their phone tests are fixed and merged (D88, D91, D93 to D95). The public server went live on 2026-10-20 (D106): https://app.praetorwallet.com on a Google Cloud VM in Hong Kong with a Let's Encrypt certificate, on the Base deployment of 2026-10-19 and its own FISCO consortium, bootstrapped with real Pyth prices (tXAU 4,132.30 USD, tXAG 56.89 USD). Its two TEEs run in real Intel TDX CVMs on Phala Cloud (agent-signer and soter-attester, dstack 0.5.12): the bank verified the attester's quote through Phala's verifier and activated it on Base, and the agent keys on the server come from the agent-signer CVM; the dstack simulator is used only in local and fork runs. On the server, with an iPhone, an account linked WeChat, a payment to a new payee was refused on-chain (UNKNOWN_PAYEE), approved with Face ID inside WeChat (SOTER, checked with Tencent by the attester in its CVM) and executed once (docs/EVIDENCE-SERVER.md). Still to come, in Phase C with the human: the WeChat Official Account messages, the EUDI wallet scan, the demo video, the Android fingerprint, web push on a phone, the pentest and the held-out red-team cases. The I19 scan of Base data is built (`make i19-scan`, D102) and found no personal data on the M16 deployment, nor on the local fork after the final red-team runs. Details: docs/PROGRESS.md.

Licence

Copyright © 2026 the Praetor team. All rights reserved. Source made available for evaluation by FinTechathon 2026 organisers and judges (LICENSE). Third-party components keep their own licences: NOTICE, THIRD_PARTY_LICENSES.md.